

Vulnerability Analysis of Ethereum Smart Contracts Under Code Obfuscation: Dataset, Methodology, and Experimental Setup

III. Dataset Collection and Experimental Methodology

A. Overview and Motivation

The security of Ethereum smart contracts has emerged as a critical concern in the blockchain ecosystem, as vulnerabilities in deployed contracts have led to substantial financial losses and erosion of user trust. While considerable research has investigated automated vulnerability detection in smart contracts, comparatively little attention has been directed toward understanding how source-level code transformations—specifically obfuscation—affect the reliability of existing analysis tools. This work addresses that gap by constructing a structured experimental framework that enables systematic, comparative evaluation of multiple static and symbolic analysis tools across both original and obfuscated Solidity contracts.

The central motivation is twofold. First, obfuscated smart contracts are increasingly encountered in adversarial deployment scenarios, where contract authors may deliberately obscure logic to evade automated auditing. Second, the robustness of widely used analysis tools under such structural perturbations remains poorly characterized. By evaluating tool behavior across both versions of the same contract, we aim to expose performance differentials that are invisible when original contracts alone are considered.

B. Dataset Collection Methodology

B.1 Source of Contract Addresses

The Ethereum smart contract addresses used in this study were collected from the publicly available DiSCo dataset repository developed by Xu et al. (2024) [1], which provides a curated collection of Ethereum contract addresses well-suited for security research. Starting from this address pool, contracts were organized into five independent batches, designated Batch 01 through Batch 05, with each batch capped at 200 contracts. This batched structure was adopted deliberately to facilitate incremental collection, parallel downstream processing, and reproducibility across experimental runs. For each batch, the candidate addresses were stored in a plain-text file named `addresses.txt`, where each line contains a single Ethereum address in standard hexadecimal format, providing a minimal and pipeline-friendly input representation.

B.2 Automated Data Collection via the Etherscan API

To retrieve the contract artifacts corresponding to each address, a dedicated Python-based collection script was developed. The script targets the Etherscan V2 API, which exposes

programmatic access to on-chain contract data for verified Ethereum mainnet deployments (chain ID: 1). Two API endpoints were used in combination. The `getsourcecode` endpoint, invoked under the `contract` module, returns the verified Solidity source code together with compilation metadata including the contract name, compiler version, EVM target version, optimization flag, number of optimization runs, and SPDX license type. The `eth_getCode` endpoint, invoked under the `proxy` module with the `latest` tag, retrieves the deployed EVM bytecode as a hexadecimal string directly from the blockchain state.

The collection workflow is structured as follows. The script reads all addresses from the batch's `addresses.txt` file and processes them sequentially. For each address, it first calls the `getsourcecode` endpoint and inspects the response. If the API returns an error string or an empty result—indicating a rate-limit hit or an inaccessible contract—the address is skipped after a short wait. If the returned `SourceCode` field is empty, the contract is classified as unverified and excluded, since source-level transparency is a prerequisite for the static and symbolic analysis tools used in subsequent experiments. Only contracts that pass this verification check proceed to the bytecode retrieval step, after which all three artifacts are written to disk. A fixed inter-request delay of 1.2 seconds is enforced between API calls throughout the script, keeping request frequency within the bounds of the Etherscan free-tier rate limit and preventing throttling errors. Collection terminates once 200 verified contracts have been saved for a given batch, or once all addresses in the file have been exhausted.

For each saved contract, the metadata recorded in `meta.txt` captures the contract name, compiler version string (e.g., `v0.8.17+commit.8df45f5f`), EVM version target, optimization status with the number of optimization runs, and the declared software license. This metadata is preserved alongside the source and bytecode to ensure that any tool requiring compiler-specific configurations can be informed appropriately during analysis.

B.3 Dataset Structure and Storage Format

Each collected contract is stored in a self-contained subdirectory within its corresponding batch directory, following the naming convention `contract_NNN/`, where `NNN` is a zero-padded three-digit index. Within each contract directory, three files are produced:

- `source.sol` — the complete verified Solidity source code as returned by the Etherscan API.
- `bytecode.txt` — the deployed EVM bytecode in hexadecimal encoding, representing the contract's on-chain executable form.
- `meta.txt` — a structured plain-text record of the compiler version, EVM version, optimization configuration, and license type.

This three-file layout maintains a clean separation of concerns and allows each stage of the downstream analysis pipeline to access only the artifact it requires. The directory tree for a representative batch takes the following form:

```
batch_XX/  
  addresses.txt  
  contract_001/  
    └─ source.sol  
    └─ bytecode.txt  
    └─ meta.txt  
  contract_002/  
    └─ source.sol  
    └─ bytecode.txt  
    └─ meta.txt  
  ...  
  contract_200/  
    └─ source.sol  
    └─ bytecode.txt  
    └─ meta.txt
```

B.4 Dataset Publication and Collaborative Access

Upon completion of each collection batch, the assembled dataset was published to a shared GitHub repository to facilitate access by all members of the research group engaged in subsequent experimental tasks. The dataset is publicly available at:

┆ <https://github.com/THIKMANIK/Smart-contract-dataset/tree/main/dataset>

This repository serves as the single source of truth for all five batches, enabling team members working on obfuscation generation and vulnerability analysis to operate from an identical, version-controlled corpus. Hosting the dataset on GitHub also ensures that the experimental inputs are traceable and that results reported in this paper can be independently reproduced by external researchers.

References

[1] J. Xu et al., "DiSCo: The DiSCo Decompiler Lifting EVM Bytecode to Source Code using LLMs," GitHub Repository, 2024. Available: <https://github.com/suxnju/disco>